Neurons and Activation Functions

An artificial neuron computes a weighted sum of its inputs, adds a bias term, and passes the result through an activation function. Without a non-linear activation function, a stack of layers would collapse into a single linear transformation. The ReLU activation, defined as the maximum of zero and the input, is widely used in hidden layers because it is cheap to compute and reduces the vanishing gradient problem. The sigmoid squashes values into the range zero to one and is often used for binary outputs, while softmax turns a vector of scores into a probability distribution over classes.

Training with Backpropagation

Neural networks are trained by minimising a loss function. Backpropagation applies the chain rule of calculus to compute the gradient of the loss with respect to every weight, moving backwards from the output layer. Gradient descent then updates each weight by subtracting the learning rate multiplied by its gradient. A learning rate that is too large makes training diverge, while one that is too small makes it very slow. Stochastic gradient descent estimates the gradient from a small mini-batch of examples, and the Adam optimizer adapts the step size for each weight using running averages of past gradients.

Convolutional and Recurrent Networks

Convolutional neural networks (CNNs) are designed for grid-like data such as images. They apply small learnable filters that slide across the input, so the same pattern detector is reused at every position, and pooling layers reduce spatial size. Recurrent neural networks (RNNs) process sequences by carrying a hidden state from one step to the next, which lets them use earlier context. Plain RNNs struggle with long sequences because gradients vanish, which motivated gated designs such as the LSTM. Transformers replaced recurrence with self-attention, allowing all positions in a sequence to be processed in parallel.